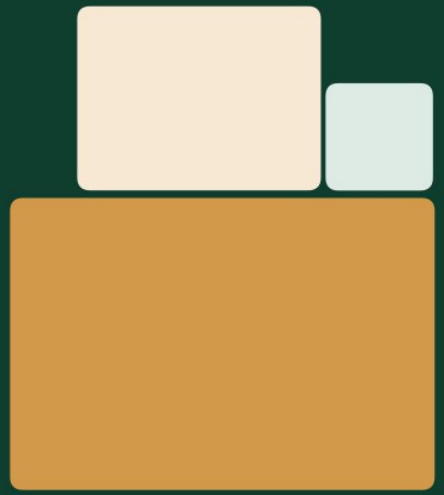
Rassah

رصّة

Type an order number. Get the cheapest safe carton, numbered packing instructions and a 3D layout, from messy product pages.

AI packing assistant for Saudi e-commerce



Project 1: AI Product Development and Monitoring · Tuwaiq Academy × Startupbootcamp × Meta · 22 Oct 2026
Team: M1 AI & Data · M2 Engine & UI · M3 Platform & Monitoring

Packers pick cartons by eye

1

Oversized cartons cost money

Couriers bill the larger of actual and volumetric weight (L × W × H / 5000). A light order in a big box pays for air.

2

Fragile items break, liquids leak

A new packer puts a book on a perfume bottle, or lays an oil bottle flat. Breakage means returns.

3

The data doesn't exist

Cartonization tools assume clean dimensions in a warehouse system. Small Saudi merchants only have free-text product pages in Arabic and English.

Problem size is a hypothesis we have not measured. Merchant outreach: docs/discovery.md.

Before and after Rassah

Before

- Packer guesses the carton
- Product sizes unknown or buried in text
- New packers learn by breaking things
- No record of what went wrong

After

- Packer types an order number
- Gemini reads each product page once (cached)
- Cheapest valid carton + numbered Arabic steps + 3D view
- Packer confirms or corrects; every correction is measured

How it works

1 LLM

Gemini understands

Product text → outer dimensions, weight, fragile / liquid / sensitive flags, source of each value, evidence quote

→

2 Search

Engine computes

3D packing search: corner grid × rotations × 9 strategies + 40 seeded restarts, per carton

→

3 Geometry

Validator proves

Independent code: no overlap, ≥ 70% base support, nothing on fragile, liquids upright, weight limit

The LLM understands, the engine computes, the validator proves.

Output: the cheapest valid carton by estimated carton + shipping cost. If nothing fits, Rassah says so.

AI extraction with honesty rules

Every value carries its source

page — Stated on the product page, with the evidence quote

inferred — Estimated from clues (50 ml, 400 pages). Gets a 15% safety margin and an “estimated” badge

unknown — No basis. The order stops: “needs measurement”

The 15% margin will be checked against the measured inferred-size error (extraction eval pending).

Rules the system enforces

- Never invent a value or a layout
- JSON schema + Zod check on every Gemini response
- Product text is passed as data, not instructions
- One extraction per product, cached; packer corrections are never overwritten
- Gemini failure → visible “extraction failed”, logged, alerted

Live demo

1001	Single book → smallest suitable carton
1002	Perfume with no dimensions: inferred, padded, upright, on top. Numbered steps + 3D slider
Edit	Packer corrects a value → recorded as a correction (metric M1)
1020	≈ 20.5 kg: every carton too small or too weak. Rassah says so
/monitoring	Correction rate, Gemini failures, p95 latency; click a run for its trace

Backup if the live demo fails: recorded video docs/report/demo.mp4 and screenshots in docs/report/screens/ (to be captured before Thursday).

Packing evaluation: 60 seeded fixtures

35 / 35

engine layouts pass the independent validator

0 / 20

false “packed” claims on impossible cases

15 / 20

known-feasible recall on zero-slack exact fills (up from 3 / 20)

27 / 35

naive-baseline layouts break a safety rule

Extraction eval: pending real Gemini runs. No cost savings claimed: 0 SAR vs baseline on the 8 fixtures where both are valid. Source: eval/results/packing-2026-10-20-engine-0.1.json

Monitoring: seven metrics, one trace per run

Dashboard screenshot
(capture from /monitoring on the deployed app)

- **M1** AI correction rate
- **M2** Data-source mix
- **M3** Validator rejections (must be 0)
- **M4** No-carton rate
- **M5** p50 / p95 latency, cached vs uncached
- **M6** Gemini failure rate
- **M7** Cost per order + cache hits

Investigation: the eval showed recall 3 / 20 → traced fixture feasible-05 (every score preferred flat items) → added “tight” scores → 15 / 20, validity still 100%, regression test added.

Limits, and what we ask for next

What it does not do yet

- Synthetic data (30 products, written by the team)
- Rigid box-shaped products only, one carton per order
- Illustrative costs, always labeled “estimated”
- Heuristic search: not globally optimal (5 / 20 exact fills missed)
- No crush-strength or void-fill modeling; no live Salla / Zid link

The ask: a pilot with one merchant

100 real orders: the packer and Rassah side by side
Measured: carton spend, courier invoices, damage, packing time
Then: Salla / Zid integration, multi-carton orders, scale-weight feedback

Team and who presents what

M1

AI & Data

Gemini extraction, synthetic dataset, extraction eval

Slides 1–5 · 0:00–1:00

M2

Engine & UI

Packing engine, validator, result page, 3D view

Slides 6–7 · 1:00–2:20 (live demo)

M3

Platform & Monitoring

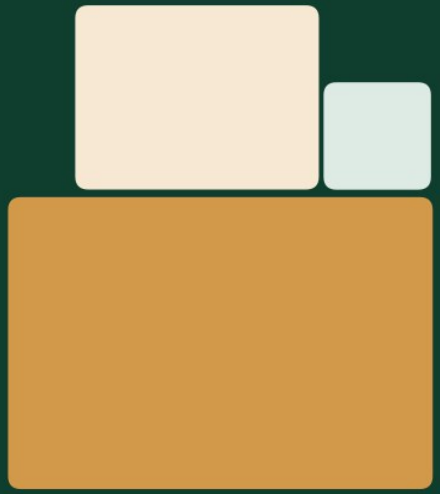
Next.js, Supabase, events, dashboard, alerts, CI

Slides 8–11 · 2:20–3:00

Every member can explain every part: each one walks through code another member wrote.

Thank you

شكراً

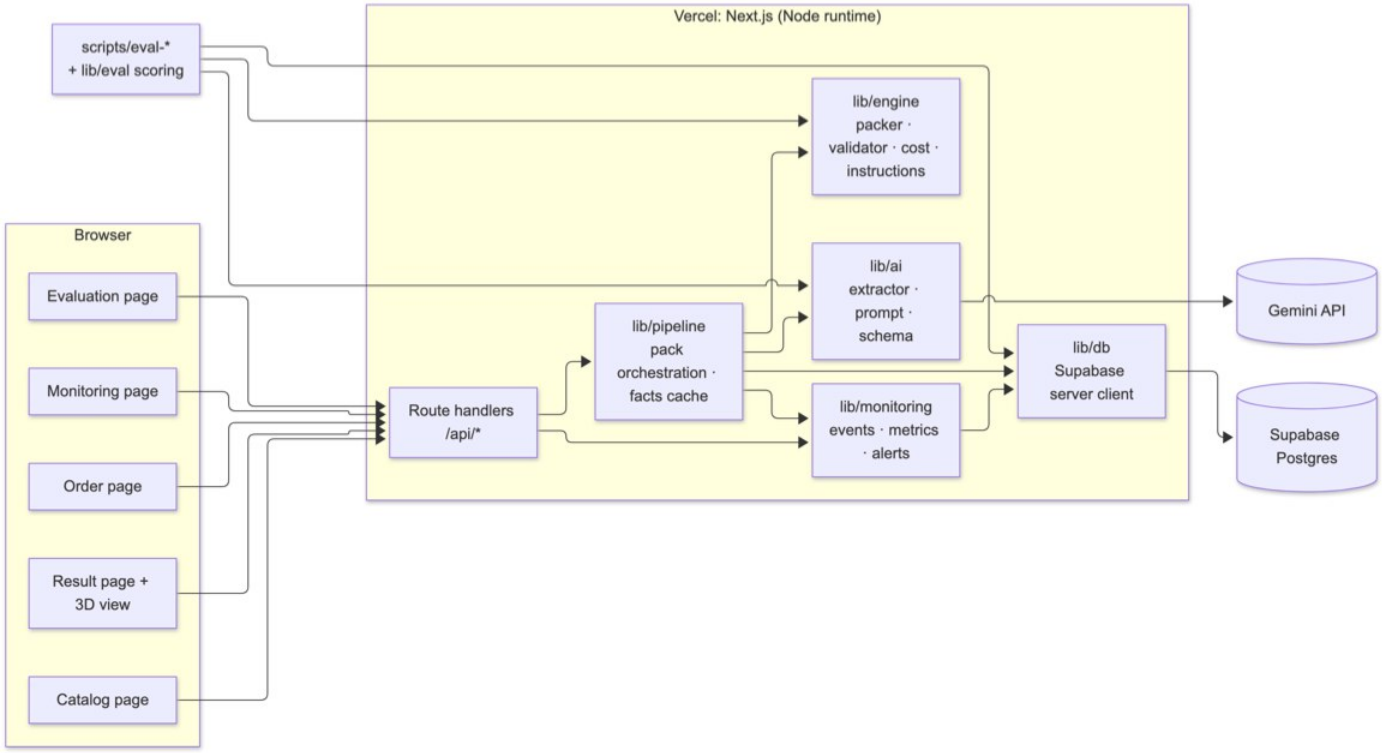


Repository **github.com/justAbdulaziz10/Rassah**

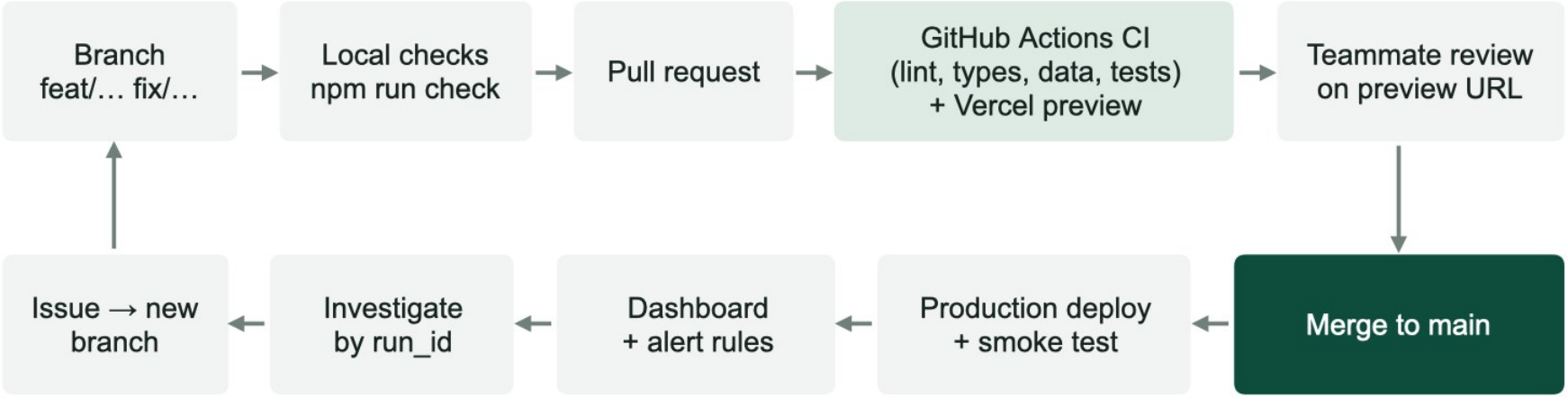
Live app **LIVE_URL_TBD**

Questions welcome. Appendix: architecture, development flow, monitoring flow, metrics, tools.

Appendix: system architecture

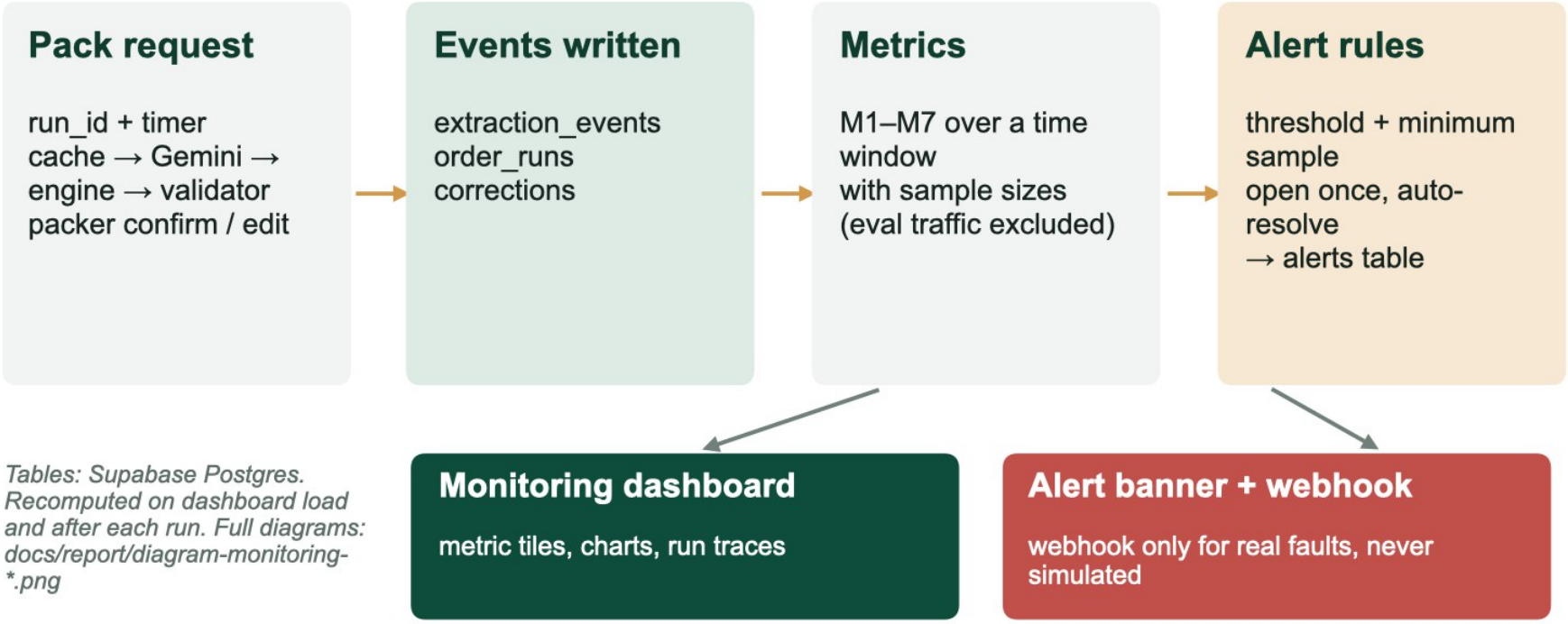


Appendix: development flow



Prompt or engine change → eval scripts (eval-extraction, eval-packing) → eval_runs in Supabase → evaluation page + report numbers, compared before merging.

Appendix: monitoring flow



Tables: Supabase Postgres. Recomputed on dashboard load and after each run. Full diagrams: docs/report/diagram-monitoring-.png*

Appendix: metric definitions and alerts

#	Metric	Reveals	Alert rule
M1	AI correction rate (reviewed AI products the packer corrected)	Wrong extractions	> 20% of last 30, warning
M2	Data-source mix of packed units	Missing store data	> 30% unknown in 24 h, info
M3	Validator rejections of engine layouts	Engine bugs (must be 0)	Any in 24 h, critical
M4	No-carton rate + skip reasons	Catalog / margin mismatch	> 15% of last 20, warning
M5	p50 / p95 latency, cached vs uncached	Slow Gemini vs slow engine / DB	p95 > 8 s uncached / 2 s cached
M6	Gemini failure rate by status	Key, quota, outage, bad output	> 10% of last 20, critical
M7	Cost per order + cache-hit rate	Cache broken, prompt bloat	Shown, no alert

Each metric shows its sample size and reads “insufficient data” below its minimum.

Appendix: tools and AI usage

Tool	Used for	Origin	Verified by
Gemini API (Flash tier)	Runtime product extraction	Product feature	Extraction eval vs ground truth (pending), Zod checks
Claude Code	Code drafts, docs, synthetic data, tests	Generated, team-reviewed	107 tests, CI, evals, end-to-end run
Next.js 16 + TypeScript	UI + API in one project	Framework	Typecheck, build
Supabase Postgres	Catalog cache, events, eval results	Service	RLS, end-to-end run
Vercel + GitHub Actions	Hosting, previews, CI	Service	CI on every PR
Vitest · R3F · Recharts	Tests, 3D view, charts	Libraries	Tests, screenshots

Full log with limitations per tool: docs/report/TOOLS.md